BLS 암호학 구현 분석

개요

이 문서는 Lighthouse의 BLS (Boneh-Lynn-Shacham) 암호학 구현을 상세히 분석합니다. BLS는 Ethereum 2.0에서 검증자 서명과 집계를 위해 사용되는 핵심 암호학 시스템으로, Lighthouse는 `crypto/bls` 모듈을 통해 이를 구현합니다.

Lighthouse의 BLS 구현은 다음과 같은 특징을 가집니다:

- **다중 백엔드 지원**: BLST (고성능) 및 fake_crypto (테스트용) 백엔드
- **제네릭 설계**: 백엔드에 독립적인 공통 인터페이스 제공
- **Ethereum 2.0 특화**: 합의 프로토콜에 최적화된 기능들
- **안전성 우선**: 무한대 점, 영 키 등 예외 상황 처리

BLS 라이브러리 아키텍처

전체 구조

```
crypto/bls/
├── src/
| ├── lib.rs # 메인 라이브러리 진입점
| ├── generic_secret_key.rs # 비밀키 제네릭 구현
| ├── generic_public_key.rs # 공개키 제네릭 구현
| ├── generic_signature.rs # 서명 제네릭 구현
| ├── generic_aggregate_signature.rs # 집계 서명 제네릭 구현
```

```
| ├── generic_keypair.rs # 키페어 제네릭 구현
| └── impls/
| ├── blst.rs # BLST 백엔드 구현
| └── fake_crypto.rs # 테스트용 가짜 암호화
└── tests/
└── tests.rs # 통합 테스트
```

백엔드 시스템

Lighthouse는 컴파일 타임 피처 플래그를 통해 다양한 BLS 백엔드를 지원합니다:

```rust
// crypto/bls/src/lib.rs
#[cfg(feature = "supranational")]
define_mod!(blst_implementations, crate::impls::blst::types);

#[cfg(feature = "fake_crypto")]
define_mod!(fake_crypto_implementations, crate::impls::fake_crypto::types);

#[cfg(all(feature = "supranational", not(feature = "fake_crypto")))]
pub use blst_implementations::*;
```

지원되는 백엔드들:

- `supranational` : BLST 라이브러리 기반 고성능 구현
- `fake_crypto` : 테스트용 항상-유효 반환 구현

핵심 BLS 컴포넌트 분석

1. 비밀키 (SecretKey) 구현

GenericSecretKey 구조체

```
// crypto/bls/src/generic_secret_key.rs
pub struct GenericSecretKey<Sig, Pub, Sec> {
/// 실제 암호화 연산을 수행하는 기본 점
point: Sec,
_phantom_signature: PhantomData<Sig>,
_phantom_public_key: PhantomData<Pub>,
}
```

주요 메서드들

키 생성:

```
impl<Sig, Pub, Sec> GenericSecretKey<Sig, Pub, Sec> {
/// 안전한 엔트로피 소스에서 Self를 인스턴스화
pub fn random() -> Self {
Self {
point: Sec::random(),
_phantom_signature: PhantomData,
_phantom_public_key: PhantomData,
}
```

```
}

}
```

서명 생성:

```
/// 메시지에 서명

pub fn sign(&self, msg: Hash256) -> GenericSignature<Pub, Sig> {

let is_infinity = false;

GenericSignature::from_point(self.point.sign(msg), is_infinity)

}
```

공개키 도출:

```
/// 자신에 대응하는 공개키 반환

pub fn public_key(&self) -> GenericPublicKey<Pub> {

GenericPublicKey::from_point(self.point.public_key())

}
```

직렬화/역직렬화:

```
/// 압축된 바이트로 직렬화

pub fn serialize(&self) -> ZeroizeHash {

self.point.serialize()

}
```

```
/// 압축된 바이트에서 역직렬화
pub fn deserialize(bytes: &[u8]) -> Result<Self, Error> {
if bytes.len() != SECRET_KEY_BYTES_LEN {
Err(Error::InvalidSecretKeyLength {
got: bytes.len(),
expected: SECRET_KEY_BYTES_LEN,
})
} else if bytes.iter().all(|b| *b == 0) {
// 모든 바이트가 0인 비밀키는 무효
Err(Error::InvalidZeroSecretKey)
} else {
Ok(Self {
point: Sec::deserialize(bytes)?,
_phantom_signature: PhantomData,
_phantom_public_key: PhantomData,
})
}
}
```

2. 공개키 (PublicKey) 구현

GenericPublicKey 구조체

```
// crypto/bls/src/generic_public_key.rs
```

```rust
pub struct GenericPublicKey<Pub> {
/// 실제 암호화 연산을 수행하는 기본 점
point: Pub,
}
```

상수 정의

```rust
/// 압축된 형태로 직렬화된 BLS 공개키의 바이트 길이
pub const PUBLIC_KEY_BYTES_LEN: usize = 48;

/// 비압축된 형태로 직렬화된 BLS 공개키의 바이트 길이
pub const PUBLIC_KEY_UNCOMPRESSED_BYTES_LEN: usize = 96;

/// 무한대 점의 공개키 표현
pub const INFINITY_PUBLIC_KEY: [u8; PUBLIC_KEY_BYTES_LEN] = [
0xc0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
];
```

주요 메서드들

직렬화:

```rust
/// 압축된 바이트로 직렬화
pub fn serialize(&self) -> [u8; PUBLIC_KEY_BYTES_LEN] {
```

```
self.point.serialize()
}

/// 비압축된 바이트로 직렬화
pub fn serialize_uncompressed(&self) -> [u8; PUBLIC_KEY_UNCOMPRESSED_BYTES_LEN
self.point.serialize_uncompressed()
}
```

역직렬화:

```
/// 압축된 바이트에서 역직렬화
pub fn deserialize(bytes: &[u8]) -> Result<Self, Error> {
if bytes == &INFINITY_PUBLIC_KEY[..] {
// 무한대 공개키는 무효
Err(Error::InvalidInfinityPublicKey)
} else {
Ok(Self {
point: Pub::deserialize(bytes)?,
})
}
}
```

3. 서명 (Signature) 구현

GenericSignature 구조체

```rust
// crypto/bls/src/generic_signature.rs
pub struct GenericSignature<Pub, Sig> {
/// 실제 암호화 연산을 수행하는 기본 점
point: Option<Sig>,
/// 이 점이 INFINITY_SIGNATURE와 같은지 여부
pub(crate) is_infinity: bool,
_phantom: PhantomData<Pub>,
}
```

상수 정의

```rust
/// 압축된 형태로 직렬화된 BLS 서명의 바이트 길이
pub const SIGNATURE_BYTES_LEN: usize = 96;

/// 무한대 점의 서명 표현
pub const INFINITY_SIGNATURE: [u8; SIGNATURE_BYTES_LEN] = [
0xc0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
// ... (96바이트 총)
];

/// "빈" 서명을 나타내는 압축된 바이트 (모든 바이트가 0)
```

```
pub const NONE_SIGNATURE: [u8; SIGNATURE_BYTES_LEN] = [0; SIGNATURE_BYTES_LEN];
```

주요 메서드들

빈 서명 생성:

```
/// "빈" 값으로 초기화. 이 값은 모든 0으로 직렬화됨
pub fn empty() -> Self {
Self {
point: None,
is_infinity: false,
_phantom: PhantomData,
}
}
```

서명 검증:

```
/// pubkey가 msg에 대한 서명인지 확인
pub fn verify(&self, pubkey: &GenericPublicKey<Pub>, msg: Hash256) -> bool {
if let Some(point) = &self.point {
point.verify(pubkey.point(), msg)
} else {
false
}
}
```

4. 집계 서명 (AggregateSignature) 구현

GenericAggregateSignature 구조체

```
// crypto/bls/src/generic_aggregate_signature.rs
pub struct GenericAggregateSignature<Pub, AggPub, Sig, AggSig> {
/// 실제 암호화 연산을 수행하는 기본 점
point: Option<AggSig>,
/// 이 점이 INFINITY_SIGNATURE와 같은지 여부
pub(crate) is_infinity: bool,
_phantom_pub: PhantomData<Pub>,
_phantom_agg_pub: PhantomData<AggPub>,
_phantom_sig: PhantomData<Sig>,
}
```

주요 메서드들

무한대 점 초기화:

```
/// 다른 서명들을 집계할 수 있는 무한대 값으로 초기화
pub fn infinity() -> Self {
Self {
point: Some(AggSig::infinity()),
is_infinity: true,
_phantom_pub: PhantomData,
_phantom_agg_pub: PhantomData,
```

```
_phantom_sig: PhantomData,
}
}
```

서명 집계:

```
/// 서명을 self에 집계
pub fn add_assign(&mut self, other: &GenericSignature<Pub, Sig>) {
if let Some(other_point) = other.point() {
self.is_infinity = self.is_infinity && other.is_infinity;
if let Some(self_point) = &mut self.point {
self_point.add_assign(other_point)
} else {
let mut self_point = AggSig::infinity();
self_point.add_assign(other_point);
self.point = Some(self_point)
}
}
}
```

빠른 집계 검증:

```
/// 모든 pubkeys가 msg에 서명한 집계 서명인지 검증
pub fn fast_aggregate_verify(&self, msg: Hash256, pubkeys: &[&GenericPublicKey
```

```
if pubkeys.is_empty() {
return false;
}

match self.point.as_ref() {
Some(point) => point.fast_aggregate_verify(msg, pubkeys),
None => false,
}
}
```

BLST 백엔드 구현 분석

BLST 라이브러리 통합

Lighthouse는 Supranational의 BLST 라이브러리를 기본 백엔드로 사용합니다:

```
// crypto/bls/src/impls/blst.rs
pub use blst::min_pk as blst_core;
use blst::{blst_scalar, BLST_ERROR};

/// BLS 서명에 사용되는 도메인 분리 태그
pub const DST: &[u8] = b"BLS_SIG_BLS12381G2_XMD:SHA-256_SSWU_RO_POP_";
pub const RAND_BITS: usize = 64;
```

비밀키 구현

```
impl TSecretKey<blst_core::Signature, blst_core::PublicKey> for blst_core::Secre

fn  random() -> Self {

let  rng = &mut rand::thread_rng();

let  ikm: [u8; 32] = rng.gen();

// BLST의 키 생성 함수 사용

Self::key_gen(&ikm, &[]).unwrap()

}



fn  public_key(&self) -> blst_core::PublicKey {

// 비밀키에서 공개키 도출

self.sk_to_pk()

}



fn  sign(&self, msg: Hash256) -> blst_core::Signature {

// 도메인 분리 태그와 함께 서명

self.sign(msg.as_slice(), DST, &[])

}

}
```

공개키 구현

```
impl TPublicKey for blst_core::PublicKey {
fn  serialize(&self) -> [u8; PUBLIC_KEY_BYTES_LEN] {
// 압축된 형태로 직렬화
self.compress()
}

fn  deserialize(bytes: &[u8]) -> Result<Self, Error> {
if  bytes.len() != PUBLIC_KEY_BYTES_LEN {
return Err(Error::InvalidByteLength {
got: bytes.len(),
expected: PUBLIC_KEY_BYTES_LEN,
});
}
// 키 검증 및 서브그룹 체크 포함
Self::key_validate(bytes).map_err(Into::into)
}
}
```

서명 구현

```
impl TSignature<blst_core::PublicKey> for blst_core::Signature {
fn  verify(&self, pubkey: &blst_core::PublicKey, msg: Hash256) -> bool {
// 서브그룹 체크와 함께 서명 검증
```

```
self.verify(true, msg.as_slice(), DST, &[], pubkey, false) == BLST_ERROR::BLST_S
}
}
```

집계 서명 구현

```
impl TAggregateSignature<blst_core::PublicKey, BlstAggregatePublicKey, blst_core
for BlstAggregateSignature
{
fn  fast_aggregate_verify(
&self,
msg: Hash256,
pubkeys: &[&GenericPublicKey<blst_core::PublicKey>],
) -> bool {
let  pubkeys = pubkeys.iter().map(|pk| pk.point()).collect::<Vec<_>>();
let  signature = self.0.clone().to_signature();
// 빠른 집계 검증 (모든 공개키가 같은 메시지에 서명)
signature.fast_aggregate_verify(true, msg.as_slice(), DST, &pubkeys)
== BLST_ERROR::BLST_SUCCESS
}
}
```

서명 세트 일괄 검증

Lighthouse는 성능 최적화를 위해 여러 서명을 한 번에 검증하는 기능을 제공합니다:

```
// crypto/bls/src/impls/blst.rs
pub fn verify_signature_sets<'a>(
signature_sets: impl ExactSizeIterator<Item = &'a SignatureSet<'a>>,
) -> bool {
let sets = signature_sets.collect::<Vec<_>>();

if sets.is_empty() {
return false;
}

let rng = &mut rand::thread_rng();
let mut rands: Vec<blst_scalar> = Vec::with_capacity(sets.len());
let mut msgs_refs = Vec::with_capacity(sets.len());
let mut sigs = Vec::with_capacity(sets.len());
let mut pks = Vec::with_capacity(sets.len());

for set in &sets {
// 랜덤 스칼라 생성 (0이 아닌 값)
let mut vals = [0u64; 4];
while vals[0] == 0 {
```

```
vals[0] = rng.gen();
}
let mut rand_i = std::mem::MaybeUninit::<blst_scalar>::uninit();

unsafe {
blst::blst_scalar_from_uint64(rand_i.as_mut_ptr(), vals.as_ptr());
rands.push(rand_i.assume_init());
}

// 메시지 슬라이스 준비
msgs_refs.push(set.message.as_slice());

if let Some(point) = set.signature.point() {
// 서명 서브그룹 체크
if !point.0.subgroup_check() {
return false;
}
sigs.push(point.0.to_signature())
} else {
return false;
}

// 공개키들을 집계
```

```
let  signing_keys = set
.signing_keys
.iter()
.map(|pk| pk.point())
.collect::<Vec<_>>();


let Ok(agg_pk) = blst_core::AggregatePublicKey::aggregate(&signing_keys, false)
return false;
};
pks.push(agg_pk.to_public_key());
}


let (sig_refs, pks_refs): (Vec<_>, Vec<_>) = sigs.iter().zip(pks.iter()).unzip()


// 다중 집계 서명 검증
let  err = blst_core::Signature::verify_multiple_aggregate_signatures(
&msgs_refs, DST, &pks_refs, false, &sig_refs, false, &rands, RAND_BITS,
);


err == blst::BLST_ERROR::BLST_SUCCESS
}
```

키페어 (Keypair) 관리

GenericKeypair 구조체

```
// crypto/bls/src/generic_keypair.rs
pub struct GenericKeypair<Pub, Sec, Sig> {
pub pk: GenericPublicKey<Pub>,
pub sk: GenericSecretKey<Sig, Pub, Sec>,
_phantom: PhantomData<Sig>,
}
```

주요 메서드들

랜덤 키페어 생성:

```
/// 랜덤하게 생성된 비밀키에서 Self를 인스턴스화
pub fn random() -> Self {
let sk = GenericSecretKey::random();
Self {
pk: sk.public_key(),
sk,
_phantom: PhantomData,
}
}
```

컴포넌트에서 생성:

```
/// 공개키와 비밀키에서 Self를 인스턴스화
pub fn from_components(pk: GenericPublicKey<Pub>, sk: GenericSecretKey<Sig, Pu
Self {
pk,
sk,
_phantom: PhantomData,
}
}
```

에러 처리 및 보안 고려사항

에러 타입 정의

```
// crypto/bls/src/lib.rs
#[derive(Clone, Debug, PartialEq)]
pub enum Error {
/// Supranational BLST BLS 라이브러리에서 발생한 에러
#[cfg(feature = "supranational")]
BlstError(BlstError),
/// 제공된 바이트의 길이가 잘못됨
InvalidByteLength { got: usize, expected: usize },
```

```
/// 제공된 비밀키 바이트의 길이가 잘못됨
InvalidSecretKeyLength { got: usize, expected: usize },
/// 공개키가 무한대 점을 나타내므로 무효
InvalidInfinityPublicKey,
/// 비밀키가 모든 0 바이트이므로 무효
InvalidZeroSecretKey,
}
```

보안 검증

무한대 점 검증:

```
// 공개키 역직렬화 시 무한대 점 체크
if  bytes == &INFINITY_PUBLIC_KEY[..] {
Err(Error::InvalidInfinityPublicKey)
}
```

영 비밀키 검증:

```
// 비밀키 역직렬화 시 모든 0 바이트 체크
if  bytes.iter().all(|b| *b == 0) {
Err(Error::InvalidZeroSecretKey)
}
```

서브그룹 검증:

```
// BLST 구현에서 서브그룹 체크
if !point.0.subgroup_check() {
return false;
}
```

실제 사용 예시

기본적인 서명 및 검증

```
use bls::*;

// 1. 키페어 생성
let  keypair = Keypair::random();
let  secret_key = &keypair.sk;
let  public_key = &keypair.pk;

// 2. 메시지 서명
let  message = Hash256::from_low_u64_be(42);
let  signature = secret_key.sign(message);

// 3. 서명 검증
assert!(signature.verify(public_key, message));
```

집계 서명 생성 및 검증

```rust
// 1. 여러 키페어 생성
let keypairs: Vec<Keypair> = (0..3).map(|_| Keypair::random()).collect();
let message = Hash256::from_low_u64_be(42);

// 2. 개별 서명 생성
let signatures: Vec<Signature> = keypairs
.iter()
.map(|kp| kp.sk.sign(message))
.collect();

// 3. 서명 집계
let mut agg_sig = AggregateSignature::infinity();
for sig in &signatures {
agg_sig.add_assign(sig);
}

// 4. 공개키 수집
let public_keys: Vec<&PublicKey> = keypairs.iter().map(|kp| &kp.pk).collect();

// 5. 집계 서명 검증
assert!(agg_sig.fast_aggregate_verify(message, &public_keys));
```

서명 세트 일괄 검증

```
use bls::SignatureSet;


// 서명 세트 생성
let signature_sets: Vec<SignatureSet> = keypairs
.iter()
.zip(signatures.iter())
.map(|(kp, sig)| SignatureSet {
signature: sig,
signing_keys: vec![&kp.pk],
message,
})
.collect();


// 일괄 검증
let signature_set_refs: Vec<&SignatureSet> = signature_sets.iter().collect();
assert!(verify_signature_sets(signature_set_refs.into_iter()));
```

성능 최적화 기법

1. 일괄 검증 (Batch Verification)

여러 서명을 개별적으로 검증하는 대신 한 번에 검증하여 성능을 향상시킵니다:

```
// 개별 검증 (느림)
for (sig, pk, msg) in signatures.iter().zip(public_keys.iter()).zip(messages.it
assert!(sig.verify(pk, *msg));
}

// 일괄 검증 (빠름)
assert!(verify_signature_sets(signature_sets.into_iter()));
```

2. 집계 서명 (Aggregate Signatures)

여러 서명을 하나로 집계하여 저장 공간과 검증 시간을 절약합니다:

```
// 개별 서명 저장 (비효율적)
let individual_sigs: Vec<Signature> = signatures;

// 집계 서명 저장 (효율적)
let mut aggregate = AggregateSignature::infinity();
for sig in signatures {
aggregate.add_assign(&sig);
}
```

3. 서브그룹 체크 최적화

BLST는 신뢰할 수 있는 소스에서 온 키에 대해 서브그룹 체크를 건너뛸 수 있습니다:

```
// 신뢰할 수 있는 키의 경우 서브그룹 체크 생략
signature.verify(false, msg.as_slice(), DST, &[], pubkey, false)
```

Ethereum 2.0 특화 기능

1. 도메인 분리 (Domain Separation)

Ethereum 2.0 사양에 따른 도메인 분리 태그 사용:

```
pub const DST: &[u8] = b"BLS_SIG_BLS12381G2_XMD:SHA-256_SSWU_RO_POP_";
```

2. 빈 서명 처리

Ethereum 2.0에서 요구하는 “빈” 서명 (모든 0 바이트) 처리:

```
pub fn empty() -> Self {
Self {
point: None,
is_infinity: false,
_phantom: PhantomData,
}
}
```

3. 무한대 점 처리

집계 서명에서 빈 집합을 나타내는 무한대 점 처리:

```
pub fn eth_fast_aggregate_verify(
&self,
msg: Hash256,
pubkeys: &[&GenericPublicKey<Pub>],
) -> bool {
if pubkeys.is_empty() && self.is_infinity() {
true // 빈 공개키 집합과 무한대 서명은 유효
} else {
self.fast_aggregate_verify(msg, pubkeys)
}
}
```

테스트 및 검증

단위 테스트 예시

```
#[test]
fn invalid_zero_secret_key() {
// 모든 0 바이트 비밀키는 무효해야 함
assert!(SecretKey::deserialize(&[0; SECRET_KEY_BYTES_LEN]).is_err());
}
```

```
#[test]
```

```rust
fn infinity_agg_sig() {
// 무한대 집계 서명 테스트
assert_eq!(
&AggregateSignature::infinity().serialize()[..],
&INFINITY_SIGNATURE[..]
);
assert!(AggregateSignature::infinity().is_infinity());
}

#[test]
fn ssz_round_trip_multiple_types() {
// SSZ 직렬화/역직렬화 라운드트립 테스트
let mut agg_sig = AggregateSignature::infinity();
ssz_round_trip(agg_sig.clone());

let msg = Hash256::from_low_u64_be(42);
let secret = secret_from_u64(42);
let sig = secret.sign(msg);
ssz_round_trip(sig.clone());
agg_sig.add_assign(&sig);
ssz_round_trip(agg_sig);
}
```

결론

Lighthouse의 BLS 암호학 구현은 다음과 같은 특징을 가집니다:

장점:

- **고성능**: BLST 백엔드를 통한 최적화된 성능
- **안전성**: 포괄적인 에러 처리 및 보안 검증
- **유연성**: 제네릭 설계를 통한 백엔드 독립성
- **Ethereum 2.0 최적화**: 합의 프로토콜 요구사항에 특화

핵심 기능:

- 키 생성, 서명, 검증의 기본 BLS 연산
- 집계 서명을 통한 효율적인 다중 서명 처리
- 일괄 검증을 통한 성능 최적화
- 도메인 분리 및 서브그룹 체크를 통한 보안 강화

이러한 구현을 통해 Lighthouse는 Ethereum 2.0 네트워크에서 요구되는 높은 성능과 보안 요구사항을 만족시키며, 검증자들이 안전하고 효율적으로 합의에 참여할 수 있도록 지원합니다.